Type-based Enforcement of Infinitary Trace Properties for Java

Serdar Erbatur[1], Ulrich Schöpp[2], and Chuangjie Xu[2]

[1] University of Texas at Dallas, USA serdar.erbatur@utdallas.edu
[2] fortiss GmbH, Munich, Germany {schoepp,xu}@fortiss.org

Abstract. The region-based type and effect system for Java by Erbatur, Hofmann and Zălinescu can be used to ensure adherence to programming guidelines by tracking event traces of terminating programs. Building on ideas of Hofmann and Chen, we extend their work to capture also *infinite* traces produced by *non-terminating* programs. We develop a type and effect system for Featherweight Java that can express properties of both finite and infinite traces, and prove its soundness with respect to the operational semantics. The system can compute information about the possible infinite traces of Featherweight Java programs. Specifically, the set of infinite traces of a method is constructed as the greatest fixed point of the operator which calculates the possible traces of method bodies. Our type inference algorithm is realized by working with the *finitary abstraction* of the system based on Büchi automata.

1 Introduction

Developing high quality software is a difficult task that requires systematic approaches to avoiding programming errors. Building on previous experience, software development processes often establish programming guidelines or policies that are designed to rule out errors that are likely to be made. For example, security guidelines may require one to sanitize all inputs from environment before further processing, they may require that all actions of some kind, such as password changes, must be recorded in a log, *etc.* While such guidelines typically capture fairly simple program properties, they help programmers to avoid making typical errors. It would be useful to enforce such guidelines statically and continuously during software development.

The relative simplicity of typical programming guidelines makes them a good target for fully automatic static analysis techniques. Simple guidelines may even be encoded into the type system. However, the type systems of programming languages like Java are not expressive enough to effectively enforce guidelines like the examples mentioned above. One option to handle such guidelines is to use software model checkers. This approach should certainly be expressive enough to handle most guidelines, but it may be limited by scalability problems. One would like to balance expressiveness and scalability.

In this paper we follow the approach of strengthening the Java type system to make it possible to verify typical programming guidelines. We follow the

approach of [5] to use an *effect type system* to capture information about the possible *program traces*. For the purposes of this paper, traces are sequences of a certain selection of *observable events* that may happen during program execution. We assume that observable events are emitted explicitly by the program. This means that we assume that our Java programs emit events by using a statement `emit(`a`)` that emits the event a, which is taken from a finite set Σ of all possible events. A trace of a program is then simply the sequence of elements of Σ that are emitted using the `emit`-statement. The effect type system extends the Java type system with annotations that statically provide information about which traces are possible when executing the code.

Let us outline the idea concretely using an example. Consider a simple server that accepts queries for various actions on private data from a user and executes them if the user is authorized to do so. A skeleton of a method implementing such a server might look as follows.

```
void serve() & A {
  while(/* user input available */) {
    ... // read next query from user
    ... // check that user is authorized for requested
        query
    emit(authcheck);
    if (/* user is authorized */) {
      ... // process private data for user
      emit(access);
    }
  }
  ... // write to log that private data was accessed
  emit(log);
}
```

We have omitted most of the actual code. What we show are `emit`-statements that record when certain interesting events happen during program execution. A trace of `serve` is now just a sequence of emitted events, such as *authcheck access authcheck log*. This trace would appear in an execution where a user first inputs a request that she is authorized for, then one she is not authorized for and then closes the connection.

The annotation "& A" on the method signature is an *effect annotation*. It gives static information about the possible traces of terminating runs of `serve`. One may think of A as a set of traces that includes all possible traces of terminating runs. For example, A could be $\{\mathit{authcheck}, \mathit{authcheck}\,\mathit{access}\}^* \cdot \{\mathit{log}\}$, *i.e.* the set of traces that are a finite repetition of *authcheck* or *authcheck access*, followed by *log*. This set would be a slight over-approximation as an effect annotation for `serve`, since the trace *log* is not actually possible.

Effect typing is a flexible basis for analyzing guidelines and other program properties. One may formalize a guideline simply as a set of traces: the set of all traces that are acceptable under the guideline. For example, one guideline might say that any access to sensitive data must be authorized. It may be approximated by the set of traces in which each *access* is immediately preceded by *authcheck*. By checking that the effect type A contains only traces from this set, the type system allows us to check `serve` for possible guideline violations. Note that we

do not expect the effect type system to perform full-scale program verification. Rather, we intend the effect type system to assist the programmer by verifying simple properties that may point to possible programming errors, in the spirit of analysis tools like Infer [4].

This idea of effect typing was developed in [5]. The type system computes effect annotations like A, which can then be used to check adherence to guidelines. Of course, it is not possible to implement the annotation A directly as a set of traces, since it may be an infinite set. The type system of [5] represents the annotations using a finite abstraction based on finite automata. It represents a guideline by the language of a finite automaton. By using different automata, one can flexibly use the type system to analyze various guidelines or other properties. The use of automata is standard in monitoring of security policies [11].

The results of [5] apply to terminating runs of programs and thus to finite traces. However, functions like `serve` also have intended non-terminating behavior. We want to verify properties, such as that *access* is immediately preceded by *authcheck*, not just of terminating runs, but also of runs where the code never leaves the `while`-loop. Such questions have led Hofmann & Chen to study how one can extend the type-based analysis methods to cover also non-terminating runs and liveness properties. We continue this line of work.

In this paper, we present a type-based analysis method for Java in the style of [5] that integrates the ideas of Hofmann & Chen [8,9] for analyzing infinitary traces. Our type system has effect annotations for the both the terminating and the non-terminating runs of the program. The above example now has an annotation of the following form.

```
void serve() & (A, B) {
  // same code as above
}
```

Here, A is an annotation for the terminating traces, as before. The annotation B is new and gives information about the non-terminating traces. The only way for `serve` to run into an infinite loop is by repeating the `while`-loop infinitely. The corresponding traces are infinite repetitions of either *authcheck* or *authcheck access*. Accordingly, a possible choice of B would be the set of these infinite sequences, *i.e.*, $\{authcheck, authcheck\ access\}^\omega$.

When analyzing infinite traces, one wants to analyze both safety properties ("nothing bad happens") and liveness properties ("something will happen eventually"). The example that *access* must be preceded by *authcheck* is an example of a safety property. An example for a liveness property would be that any access must be logged, *i.e.* that *access* must be followed by *log*. The above example code would not have this property, since there are infinite traces where no access is logged. The approach of Hofmann & Chen accounts for both safety and liveness properties by using Büchi automata to capture properties.

The contribution of this paper is to develop a combination of the Büchi type approach of [8,9] with the region-based typing approach for Java of [5]. Starting from the ideas in [8], we develop a Büchi type and effect system for Java, establish soundness and present a type inference algorithm. To formulate the type system generically, we introduce a notion of Büchi abstraction that abstracts from the

automata-theoretic construction in [8] and that captures just the assumptions needed for the type system. We refine the region typing information of [5] to allow static class analysis, see *e.g.* [3]. We give a simple characterization of the infinitary effect analysis as a single greatest fixed point, which allows us to prove soundness simply by induction on the approximation of the greatest fixed point.

2 Events and traces

Let Σ be a finite alphabet. We use Σ^* to denote the set of finite words over Σ, and $\Sigma^{\leq\omega}$ the set of finite and infinite words. We write uv for the concatenation of the finite word u with the word v which can finite or infinite. We now extend concatenation to and recall some other operations on languages: Let $U, U' \subseteq \Sigma^*$ and $V \subseteq \Sigma^{\leq\omega}$. We write

- $U \cdot U' = \{uu' \mid u \in U, u' \in U'\} \subseteq \Sigma^*$,
- $U \cdot V = \{uv \mid u \in U, v \in V\} \subseteq \Sigma^{\leq\omega}$,
- $U^* = \{u_1 u_2 \dots u_n \mid n \in \mathbb{N} \text{ and } u_i \in U \text{ for all } i < n\} \subseteq \Sigma^*$,
- $U^+ = U^* \setminus \{\varepsilon\} \subseteq \Sigma^*$, and
- $U^\omega = \{u_1 u_2 \dots u_i \dots \mid u_i \in U \text{ for all } i \in \mathbb{N}\} \subseteq \Sigma^{\leq\omega}$.

We often write a^* rather than $\{a\}^*$ and similarly a^+, a^ω. Note that U^* is the least fixed point of $\lambda X.\{\varepsilon\} \cup U \cdot X$, where ε is the empty word, and U^ω the greatest fixed point of $\lambda X.L \cup (U \setminus \{\varepsilon\}) \cdot X$, where $L = U^*$ if $\varepsilon \in U$ and $L = \emptyset$ otherwise.

We instrument programs with special commands issuing *events* from Σ. Then a terminating execution of a program will generate a finite *trace*, that is, a word from Σ^*, whereas a non-terminating execution will generate a finite or infinite trace, that is, a word from $\Sigma^{\leq\omega}$. Our objective is to capture all possible traces of a given program via a type and effect system, so that we can verify whether they are allowed by a given *guideline*, that is, a language of acceptable traces.

3 Featherweight Java with updates and casts

We work with Featherweight Java [10] extended with updates and casts, and add a primitive operation $\texttt{emit}(a)$ that generates an event a from a finite alphabet Σ. The syntax of the language uses four kinds of names.

$$\begin{array}{rlrl} \text{variables:} & x, y \in \mathit{Var} & \text{fields:} & f \in \mathit{Fld} \\ \text{classes:} & C, D \in \mathit{Cls} & \text{methods:} & m \in \mathit{Mtd} \end{array}$$

Program expressions are defined as follows:

$$\begin{aligned} \mathit{Expr} \ni e ::= {} & x \mid \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \mid \texttt{if}\ x_1 = x_2\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \mid \texttt{null} \mid \texttt{new}\ C \\ & \mid (C)\, e \mid x.f \mid x_1.f := x_2 \mid x.m(\bar{y}) \mid \texttt{emit}(a) \end{aligned}$$

We assume the distinguished elements $\texttt{this} \in \mathit{Var}$ and $\texttt{Object} \in \mathit{Cls}$. We further assume a formal element $\texttt{NullType} \in \mathit{Cls}$ that plays the role of the type of `null` from the Java language specification [7, §4]. It may not be used in programs, *i.e.* we require $C \neq \texttt{NullType}$ in the expressions $\texttt{new}\ C$ and $(C)\, e$.

A Featherweight Java program $(\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ consists of

- a subtyping relation $\prec \in \mathcal{P}^{\text{fin}}(\mathit{Cls} \times \mathit{Cls})$ with $C \prec D$ meaning that C is an immediate subclass of D,
- a field list $\mathit{fields} : \mathit{Cls} \to \mathcal{P}^{\text{fin}}(\mathit{Fld})$ mapping a class to its fields,
- a method list $\mathit{methods} : \mathit{Cls} \to \mathcal{P}^{\text{fin}}(\mathit{Mtd})$ mapping a class to its methods,
- a method table $\mathit{mtable} : \mathit{Cls} \times \mathit{Mtd} \rightharpoonup \mathit{Var}^* \times \mathit{Expr}$ mapping a method to the pair of its formal parameters and its body.

All components are required to be *well-formed*. We refer the reader to *e.g.* [5, Section 3] for details. Let $\preceq$ be the reflexive and transitive closure of $\prec$. We have $C \preceq \texttt{Object}$ and $\texttt{NullType} \preceq C$ for all $C \in \mathit{Cls}$.

Example 1. As a running example, we consider the following Java code.

```
class Node {
  Node next;
  Node last() {
    emit(a);
    if (next == null) {
      return this;
    } else {
      return next.last();
    }
  }
}
```

```
class Test {
  Node linear() {
    Node x = new Node();
    Node y = new Node();
    y.next = x;
    return y.last();
  }
  Node cyclic() {
    Node z = new Node();
    z.next = z;
    return z.last();
  }
}
```

In Featherweight Java, the class `Node` is represented by

$$\begin{aligned} \mathit{fields}(\texttt{Node}) &= \{\texttt{next}\} \\ \mathit{methods}(\texttt{Node}) &= \{\texttt{last}\} \\ \mathit{mtable}(\texttt{Node}, \texttt{last}) &= ((), e_{\texttt{last}}) \end{aligned}$$

$$\begin{aligned} e_{\texttt{last}} := \ & \texttt{let } _ = \texttt{emit}(a) \texttt{ in} \\ & \texttt{let } x = \texttt{this.next} \texttt{ in} \\ & \texttt{let } y = \texttt{null} \texttt{ in} \\ & \texttt{if } x = y \texttt{ then this} \\ & \texttt{else let } z = \texttt{this.next} \texttt{ in } z.\texttt{last}() \end{aligned}$$

where () is the empty vector, as `last` has no parameters. □

In the standard type system of Featherweight Java, types are simply classes. It is modeled by a class table (F_0, M_0), where the *field typing* $F_0 : \mathit{Cls} \times \mathit{Fld} \rightharpoonup \mathit{Cls}$ assigns to each class C and each field $f \in \mathit{fields}(C)$ the class of f, and the *method typing* $M_0 : \mathit{Cls} \times \mathit{Mtd} \rightharpoonup \mathit{Cls}^* \times \mathit{Cls}$ specifies for each class C and each method $m \in \mathit{Mtd}(C)$ the classes of the argument variables and of the result value of m. The typings are required to be invariant w.r.t. subclasses of C (see *e.g.* [10]).

The operational semantics uses a notion of state (s, h) consisting of a *store* $s : \mathit{Var} \rightharpoonup \mathit{Val}$ that maps variables to values and a *heap* $h : \mathit{Loc} \rightharpoonup \mathit{Obj}$ that maps locations to objects. The only kinds of values are object locations or *null*. An object contains a class identifier and a valuation of its fields.

$$\begin{aligned} \text{locations:} \quad & l \in \mathit{Loc} & \qquad \text{stores:} \quad & s \in \mathit{Var} \rightharpoonup \mathit{Val} \\ \text{values:} \quad & v \in \mathit{Val} = \mathit{Loc} \uplus \{\mathit{null}\} & \qquad \text{heaps:} \quad & h \in \mathit{Loc} \rightharpoonup \mathit{Obj} \\ \text{objects:} \quad & \mathit{Obj} = \mathit{Cls} \times (\mathit{Fld} \rightharpoonup \mathit{Val}) \end{aligned}$$

The operational semantics of terminating evaluation is given as a big-step relation $(s, h) \vdash e \Downarrow v, h' \,\&\, w$ which means that, in state (s, h), the expression e evaluates to the value v with the heap updated to h', generating the event trace $w \in \Sigma^*$. The inference rules can be found in Appendix B. The operational semantics of divergence $(s, h) \vdash e \Uparrow \,\&\, w$, meaning that the evaluation of e diverges and generates the trace $w \in \Sigma^{\leq\omega}$, is defined *coinductively* from the rules in Appendix B where double horizontal lines are used.

4 Region-based type and effect system

We extend the region-based type and effect system of Erbatur *et al.* [5] to capture also infinite traces of Featherweight Java programs. For the sake of understandability, in this section we present the system with arbitrary languages over Σ as effect annotations. To make it suitable for practical use, we can work with its finitary abstraction as presented in Section 5.

In the standard type system of Featherweight Java, types are simply classes and a class table provides the typing information of fields and methods. Our system extends it in two aspects: (1) The method typing assigns to each method signature its return type as well as its *effects*, that is, the sets of possible traces (finite or infinite) that the method may generate. (2) To improve the precision of effect typing, reference types are annotated with *regions* that provide additional information about the referenced objects.

The idea of region typing is to subdivide the heap into a finite number of regions and to use the types to statically keep track of the regions that objects may belong to. This is done by refining reference types with region information. For a class C, the Java type C is the type of references to objects of class C or a subclass. This type is refined into a type of the form C_R, where R is a set of region names. It is the type of references to objects of class C or a subclass that are located in the regions mentioned by R.

There are many useful ways of subdividing the heap into regions. A simple example would be to partition the heap according to the classes of objects. For each class $C \in \mathit{Cls}$ one has one region named i_C that comprises the heap locations of all objects of actual class C. With this choice, the refinement of reference types amounts to a static approximation of the actual class of objects. For example, the type $C_{\{i_D, i_E\}}$ refines the Java reference type C with the additional information that the reference points to an object in one of the regions i_D or i_E. In other words, the referenced object must have class D or E.

This idea can be refined by partitioning the heap not just by classes, but by the allocation sites of objects. Suppose we annotate each `new` C-expression in the program with a unique label r. We can then use these labels as region names and partition the heap according to where the objects were created. A region r comprises the heap locations of all objects that were created by the particular `new` C-expression that is labelled with r. Note that all these objects must have class C, just as for i_C above. However, two objects of class C are placed in different regions if they are created at different locations in the program.

This refinement of regions is useful to improve the precision of effect typing. Our type system analyzes the effects of methods separately for each region. By analyzing effects separately for each region, we avoid accidental over-approximation of actual effects, as Example 2 below demonstrates.

Other possible choices of regions would be to distinguish between application objects and library objects or between tainted and untainted strings, for example.

In the development of the type system, we do not need to make many assumptions about the particular choice of regions. We assume that there is a finite set *Reg* of region names, which have no structure other than their identity. We assume that each region $r \in \mathit{Reg}$ comes with a set $\mathit{Cls}(r) \subseteq \mathit{Cls}$ of the classes that are allowed in the region. It is always possible to allow all classes, but more precise choices like $\mathit{Cls}(i_C) = \{C\}$ lead to more precise effect types.

4.1 Refined types

We refine the standard types of Java by annotating classes with sets of regions. The set *Typ* of *refined types* is defined as follows:

$$\mathit{Typ} \ni \tau, \sigma ::= C_R \qquad \text{where } C \in \mathit{Cls} \text{ and } R \subseteq \mathit{Reg}$$

The refined type C_R is intended to represent references to objects of (a subclass of) class C and that are located in some region in R. We call a refined type C_R *atomic* if R is a singleton. Let *ATyp* denote the set of atomic refined types. We often write C_r instead of $C_{\{r\}}$.

We define a *subtyping* relation between refined types as follows: $C_R \leq: D_S$ if and only if $C \preceq D$ and $R \subseteq S$. To formulate well-formedness of method typing, we extend the subtyping relation to sequences of types by defining $\bar{\sigma} \leq: \bar{\tau}$ if and only if $|\bar{\sigma}| = |\bar{\tau}|$ and $\sigma_i \leq: \tau_i$ for all $i \in \{1, \ldots, |\bar{\sigma}|\}$, and lift it to a partial order $\sqsubseteq$ on $\mathit{Typ} \times \mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^{\leq\omega})$ defined by $(\tau, U, V) \sqsubseteq (\tau', U', V')$ if and only if $\tau \leq: \tau'$, $U \subseteq U'$ and $V \subseteq V'$.

Given two refined types C_R and D_S, we define their join as $C_R \sqcup D_S = E_{R \cup S}$ where E is the smallest common superclass of C and D. We lift this definition to construct the join of pairs $(\tau, U), (\tau', U') \in \mathit{Typ} \times \mathcal{P}(\Sigma^*)$ as $(\tau, U) \sqcup (\tau', U') = (\tau \sqcup \tau', U \cup U')$. Given a set $L = \{(\tau_1, U_1), \ldots, (\tau_n, U_n)\} \subseteq \mathit{Typ} \times \mathcal{P}(\Sigma^*)$ for some $n \geq 0$, we write $\sqcup L$ to denote the join $(\tau_1, U_1) \sqcup \cdots \sqcup (\tau_n, U_n)$ where by convention $\sqcup\emptyset = (\texttt{NullType}_\emptyset, \emptyset)$. These join operations will be used in the typing rules.

4.2 Region-based type and effect system

In our system, the typing information for a Featherweight Java program is given by a *class table* (F, M) that consists of

- a *field typing* $F : \mathit{Fld} \times \mathit{ATyp} \rightharpoonup \mathit{Typ}$ that assigns to each atomic refined type C_r and field $f \in \mathit{methods}(C)$ the type $F(f, C_r)$ of the field f, and

– a *method typing* $M : \mathit{Mtd} \times \mathit{ATyp} \times \mathit{ATyp}^* \rightharpoonup \mathit{Typ} \times \mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^{\leq\omega})$ that assigns to each atomic refined type C_r, method $m \in \mathit{methods}(C)$ and sequence $\bar{\sigma}$ of atomic refined types for the arguments, a tuple $M(m, C_r, \bar{\sigma}) \in \mathit{Typ} \times \mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^{\leq\omega})$ specifying the refined type of the result value and the possible finite and infinite effects of the method m.

A class table is required to be *well-formed* in the following sense:

– $F(f, C_r) = F(f, D_r)$ whenever $C \preceq D$ and $f \in \mathit{fields}(D)$.
– $M(m, C_r, \bar{\sigma}) \sqsubseteq M(m, D_r, \bar{\tau})$ whenever $C \preceq D$, $m \in \mathit{methods}(D)$ and $\bar{\tau} \leq: \bar{\sigma}$.

We use Java-like notation to present the class table. An entry of the method typing table $M(m, C_r, \langle D1_{r_1}, \ldots, Dn_{r_n}\rangle) = (E_R, U, V)$ is written as:

```
class C_r {
  E_R m(D1_r1 x1, ..., Dn_rn xn) & (U,V);
}
```

It represents the type signature of method m of objects of class C in region r, with the formal arguments of types $D1_{r_1}, \ldots, Dn_{r_n}$. If the method m terminates, then it returns value of type E_R and the generated event trace is in U. If it diverges, then the generated trace is in V. Because each tuple in the domain $\mathrm{dom}(M)$ of the method typing M decides the signature of a method, we call it a *method signature*.

Example 2. A possible class table for the code from Example 1 is shown in Java notation below. It uses four distinct regions r_1, r_2, r_3 and r_4.

```
class Node_r1 {
  Node_∅ next;
  Node_r1 last() & ({a},∅);
}
class Node_r2 {
  Node_r1 next;
  Node_{r1,r2} last() & ({a,aa},∅);
}
```

```
class Node_r3 {
  Node_r3 next;
  Node_r3 last() & (a^+, a^ω);
}
class Test_r4 {
  Node_{r1,r2} linear() & ({a,aa},∅);
  Node_r3 cyclic() & (a^+, a^ω);
}
```

The members of class `Node` are given different types depending on the region. In region r_1, the field `next` has type $\texttt{Node}_\emptyset$. This means that `next` can only be `null`, since the location of any object would abstract to some region and the empty set excludes this case. As a result, method `last` will always have a as its trace, thus the effect of this method is $(\{a\}, \emptyset)$. In region r_2, the field `next` has type $\texttt{Node}_{r_1}$. Since the `last` in region r_1 only has a as a trace, we know that the traces of `last` in region r_2 can only be a or aa. This explains the effect annotation $(\{a, aa\}, \emptyset)$. In region r_3, we have a circularity, as `next` is also in r_3. □

As illustrated above, the method typing provides both the return type and the effects of each method signature. We verify its correctness by introducing an effect type system and then proving that it is sound w.r.t. the operational semantics (Theorem 1).

In our system, a typing judgment takes the form $\Gamma \vdash e : \tau \;\&\; (U, V(\mathfrak{X}))$, where $\Gamma : \mathit{Var} \rightharpoonup \mathit{Typ}$ is a typing environment, $e \in \mathit{Expr}$ a term expression, $\tau \in \mathit{Typ}$

a refined type, $U \subseteq \Sigma^*$ a set of finite traces and $V(\mathfrak{X})$ a trace expression. The set U contains all traces generated during terminating executions of e. Instead of a trace expression $V(\mathfrak{X})$, the reader may have expected a set $V \subseteq \Sigma^{\leq\omega}$ of traces generated in non-terminating executions. However, typing judgments are derived inductively while non-terminating traces cannot be captured inductively. To account for them, the type system merely generates a specification of the non-terminating behavior of e in the form of a trace expression $V(\mathfrak{X})$, which will then be subject to a coinductive treatment.

Specifically, we assign to each method signature $\delta \in \mathrm{dom}(M)$ a variable X_δ, which is intended to denote the set of non-terminating traces of the method, and write $\mathfrak{X} = \{X_\delta \mid \delta \in \mathrm{dom}(M)\}$ for the set of all such variables. The trace expression $V(\mathfrak{X})$ explains the non-terminating behavior of e under the assumption that the non-terminating behavior of all methods is given by the variables in $\mathfrak{X}$. For example, the trace expression for the term $\texttt{emit}(a); \texttt{this.f}()$ would be $\{a\} \cdot X_f$, where f is the signature of `f`.

To obtain the actual infinitary traces, we assemble the trace expressions for all methods into an equation system $\{X_\delta = V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ whose *greatest solution* gives all the possible non-terminating traces for each method (Theorem 4(2)). For example, if $\texttt{emit}(a); \texttt{this.f}()$ is the body of method `f` itself, then we get the equation $X_f = \{a\} \cdot X_f$. It expresses that non-terminating traces of `f` begin with an a and continue with a non-terminating trace of `f`. The greatest solution of this equation is a^ω, as expected.

Formally, a *trace expression* $V(\mathfrak{X})$ takes the form $\bigcup_{X \in \mathfrak{X}} (A_X \cdot X) \cup B$ where $A_X \subseteq \Sigma^*$ and $B \subseteq \Sigma^{\leq\omega}$, because concatenation and union are the only needed operations to manipulate effects. We write $\emptyset$ for the expression where $B = \emptyset$ and $A_X = \emptyset$ for all $X \in \mathfrak{X}$, and write X itself for the expression where $A_X = \{\varepsilon\}$, $B = \emptyset$ and $A_Y = \emptyset$ for all $Y \in \mathfrak{X} \setminus \{X\}$, because $\emptyset \cdot V = \emptyset$ and $\{\varepsilon\} \cdot V = V$. We define concatenation and union on expressions as follows:

- $A \cdot V(\mathfrak{X}) = \bigcup_{X \in \mathfrak{X}} ((A \cdot A_X) \cdot X) \cup (A \cdot B)$
- $V(\mathfrak{X}) \cup V'(\mathfrak{X}) = \bigcup_{X \in \mathfrak{X}} ((A_X \cup A'_X) \cdot X) \cup (B \cup B')$

where $A \subseteq \Sigma^*$ and $V'(\mathfrak{X}) = \bigcup_{X \in \mathfrak{X}} (A'_X \cdot X) \cup B'$. Given an assignment function $\eta : \mathfrak{X} \to \mathcal{P}(\Sigma^{\leq\omega})$, we obtain for each expression $V(\mathfrak{X})$ a language $V(\eta) \subseteq \Sigma^{\leq\omega}$ by substituting $\eta(X)$ for each variable X. We lift the subset relation to expressions: $V(\mathfrak{X}) \subseteq V'(\mathfrak{X})$ if and only if $V(\eta) \subseteq V'(\eta)$ for all η. Notice that any expression $V(\mathfrak{X})$ can be rewritten to $A \cdot X \cup V'(\mathfrak{X} - X)$ for some A and $V'(\mathfrak{X} - X)$, where $\mathfrak{X} - X$ abbreviates $\mathfrak{X} \setminus \{X\}$, such that $V(\eta) = A \cdot \eta(X) \cup V'(\eta)$ for any η, because union is commutative.

The typing rules are given in Figure 1. Rule PRIM says that $\texttt{emit}(a)$ has the terminating trace a and no non-terminating traces. Rules NULL, NEW and GET have similar effects. In rule IF, we have $C_{R \cap S}$ in the left premise, as this is the case where x and y are identical, which means that they must be in the same region. In rule CALL for method call, for the traces produced during terminating executions, we take the union of those computed w.r.t. all regions where the object x may reside; and similarly, we take the union of the corresponding variables to represent the possible traces of non-terminating executions. The subtyping rule SUB makes

$$\text{SUB}\ \frac{\Gamma \vdash e : \tau \ \&\ (U, V(\mathfrak{X})) \qquad \tau \leq: \tau' \qquad U \subseteq U' \qquad V(\mathfrak{X}) \subseteq V'(\mathfrak{X})}{\Gamma \vdash e : \tau' \ \&\ (U', V'(\mathfrak{X}))}$$

$$\text{PRIM}\ \frac{}{\Gamma \vdash \mathtt{emit}(a) : \mathtt{NullType}_\emptyset \ \&\ (\{a\}, \emptyset)} \qquad \text{VAR}\ \frac{}{\Gamma,\, x : \tau \vdash x : \tau \ \&\ (\{\varepsilon\}, \emptyset)}$$

$$\text{IF}\ \frac{\begin{array}{c}\Gamma,\, x : C_{R\cap S},\, y : D_{R\cap S} \vdash e_1 : \tau_1 \ \&\ (U_1, V_1(\mathfrak{X})) \\ \Gamma,\, x : C_R,\ y : D_S \vdash e_2 : \tau_2 \ \&\ (U_2, V_2(\mathfrak{X}))\end{array}}{\Gamma,\, x : C_R,\, y : D_S \vdash \mathtt{if}\ x = y\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 : \tau_1 \sqcup \tau_2 \ \&\ (U_1 \cup U_2, V_1(\mathfrak{X}) \cup V_2(\mathfrak{X}))}$$

$$\text{LET}\ \frac{\Gamma \vdash e_1 : \tau_1 \ \&\ (U_1, V_1(\mathfrak{X})) \qquad \Gamma,\, x : \tau_1 \vdash e_2 : \tau_2 \ \&\ (U_2, V_2(\mathfrak{X}))}{\Gamma \vdash \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2 : \tau_2 \ \&\ (U_1 \cdot U_2, V_1(\mathfrak{X}) \cup U_1 \cdot V_2(\mathfrak{X}))}$$

$$\text{NULL}\ \frac{}{\Gamma \vdash \mathtt{null} : \mathtt{NullType}_\emptyset \ \&\ (\{\varepsilon\}, \emptyset)} \qquad \text{NEW}\ \frac{r \in \mathit{Reg} \qquad C \in \mathit{Cls}(r)}{\Gamma \vdash \mathtt{new}\ C : C_r \ \&\ (\{\varepsilon\}, \emptyset)}$$

$$\text{CAST}\ \frac{\Gamma \vdash e : C_R \ \&\ (U, V(\mathfrak{X})) \qquad C \preceq D \text{ or } D \preceq C}{\Gamma \vdash (D)\, e : D_R \ \&\ (U, V(\mathfrak{X}))}$$

$$\text{CALL}\ \frac{(\tau, U) = \bigsqcup\{(\tau_r, U_r) \mid (\tau_r, U_r, _) = M(m, C_r, \bar\sigma), r \in R\}}{\Gamma,\, x : C_R,\, \bar y : \bar\sigma \vdash x.m(\bar y) : \tau \ \&\ (U, \bigcup_{r\in R} X_{m,C_r,\bar\sigma})}$$

$$\text{GET}\ \frac{\tau = \bigsqcup\{F(f, C_r) \mid r \in R\}}{\Gamma,\, x : C_R \vdash x.f : \tau \ \&\ (\{\varepsilon\}, \emptyset)} \qquad \text{SET}\ \frac{\tau \leq: F(f, C_r) \text{ for all } r \in R}{\Gamma, x : C_R, y : \tau \vdash x.f := y : \tau \ \&\ (\{\varepsilon\}, \emptyset)}$$

Fig. 1. Region-based type and effect system

the type system declarative in the sense that expressions can have more than one type. However, this makes it easier to relate to the Büchi type system introduced in Section 5 so that the soundness of the Büchi type system follows directly from the soundness of this declarative type system.

A program $P = (\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ is *well-typed* w.r.t. (F, M) if there exists a family $\{V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ of trace expressions such that for every $\delta = (m, C_r, \bar\sigma) \in \mathrm{dom}(M)$ with $M(\delta) = (\tau, U, V)$ and $\mathit{mtable}(C, m) = (\bar x, e)$ and $C \in \mathit{Cls}(r)$, the following hold:

1. $\mathtt{this} : C_r,\ \bar x : \bar\sigma \vdash e : \tau \ \&\ (U, V_\delta(\mathfrak{X}))$, and
2. $\eta(X_\delta) \subseteq V$,

where $\eta : \mathfrak{X} \to \mathcal{P}(\Sigma^{\leq\omega})$ is the assignment function given by the greatest solution of the equation system $\{X_\delta = V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$. We call the family $\{V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ the *associated expressions* of P, and η the *greatest solution* to P.

Example 3. Continuing the running example for the code from Example 1, we give the typing judgements for the methods in this code. With the typing tables from Example 2, the above typing rules can be used to derive the following judgements for the body $e_{\mathtt{last}}$ of `last`.

$$\begin{array}{l}\mathtt{this} : \mathtt{Node}_{r_1} \vdash e_{\mathtt{last}} : \mathtt{Node}_{r_1} \ \&\ (\{a\}, \emptyset) \\ \mathtt{this} : \mathtt{Node}_{r_2} \vdash e_{\mathtt{last}} : \mathtt{Node}_{\{r_1, r_2\}} \ \&\ (\{a, aa\}, \{a\} \cdot X_{\mathtt{last},\mathtt{Node}_{r_1},()}) \\ \mathtt{this} : \mathtt{Node}_{r_3} \vdash e_{\mathtt{last}} : \mathtt{Node}_{r_3} \ \&\ (a^+, \{a\} \cdot X_{\mathtt{last},\mathtt{Node}_{r_3},()})\end{array}$$

In the last judgement, the expression $\{a\} \cdot X_{\texttt{last},\texttt{Node}_{r_3},()}$ has the meaning that any non-terminating trace of `last` begins with a and continues with a non-terminating trace of `last` in class $\texttt{Node}_{r_3}$.

For the bodies of `linear` and `cyclic`, we have the following typings, which we can derive by giving the local variables `x`, `y` and `z` in the bodies the types $\texttt{Node}_{r_1}$, $\texttt{Node}_{r_2}$ and $\texttt{Node}_{r_3}$ respectively.

$$\texttt{this} : \texttt{Test}_{r_4} \vdash e_{\texttt{linear}} : \texttt{Node}_{\{r_1,r_2\}} \;\&\; (\{a, aa\}, X_{\texttt{last},\texttt{Node}_{r_2},()})$$
$$\texttt{this} : \texttt{Test}_{r_4} \vdash e_{\texttt{cyclic}} : \texttt{Node}_{r_3} \;\&\; (a^+, X_{\texttt{last},\texttt{Node}_{r_3},()})$$

The program from Example 1 is well-typed w.r.t. the typing (F, M) from Example 2 if we choose $\{V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ to be the family of expressions from the above typing judgements. The equation system $\{X_\delta = V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ spells out as:

$$X_{\texttt{last},\texttt{Node}_{r_1},()} = \emptyset \qquad X_{\texttt{linear},\texttt{Test}_{r_4},()} = X_{\texttt{last},\texttt{Node}_{r_2},()}$$
$$X_{\texttt{last},\texttt{Node}_{r_2},()} = \{a\} \cdot X_{\texttt{last},\texttt{Node}_{r_1},()} \qquad X_{\texttt{cyclic},\texttt{Test}_{r_4},()} = X_{\texttt{last},\texttt{Node}_{r_3},()}$$
$$X_{\texttt{last},\texttt{Node}_{r_3},()} = \{a\} \cdot X_{\texttt{last},\texttt{Node}_{r_3},()}$$

The greatest solution η of the equation system maps the variables $X_{\texttt{last},\texttt{Node}_{r_1},()}$, $X_{\texttt{last},\texttt{Node}_{r_2},()}$ and $X_{\texttt{linear},\texttt{Test}_{r_4},()}$ to $\emptyset$, and $X_{\texttt{last},\texttt{Node}_{r_3},()}$ and $X_{\texttt{cyclic},\texttt{Test}_{r_4},()}$ to a^ω. This shows that the program from Example 1 is well-typed w.r.t. the typing from Example 2. □

If a program is well-typed, then the method typing provides correct types and effects in the following sense.

Theorem 1 (Soundness). *Let P be a well-typed program w.r.t. (F, M). For any m, C, r and $\bar{\sigma}$ with $M(m, C_r, \bar{\sigma}) = (\tau, U, V)$ and for any x, $\bar{y}$, s, h, v, h' and w, we have*

1. *$x : C_r, \bar{y} : \bar{\sigma} \vdash x.m(\bar{y}) : \tau \;\&\; (U, X_{m,C_r,\bar{\sigma}})$;*
2. *if $(s, h) \vdash x.m(\bar{y}) \Downarrow v, h' \;\&\; w$ then $w \in U$;*
3. *if $(s, h) \vdash x.m(\bar{y}) \Uparrow \;\&\; w$, then $w \in V$.*

The above result is a corollary of a more general statement of soundness (Theorem 4) proved in Appendix C where we carry out the proof by induction on the approximation to the greatest solution.

5 Region-based Büchi type and effect system

The type system introduced in Section 4 is not yet suitable for practical use for two reasons: (1) Expressions are not uniquely typed. Hence, we have no type inference algorithm. (2) Effects are given as arbitrary languages. We have no direct way to compute greatest fixed points of equation systems over $\mathcal{P}(\Sigma^{\leq\omega})$.

In this section, we introduce an *algorithmic* type system (Section 5.2) where effect annotations are taken from a *finite* structure to represent the languages over Σ (Section 5.1) using the methods from abstract interpretation [2]. In Section 5.3, we present a *type inference algorithm* that infers a class table for a given program, provided a standard Java typing for the program is given.

5.1 Büchi abstraction

Instead of $\mathcal{P}(\Sigma^*)$ and $\mathcal{P}(\Sigma^{\leq\omega})$, we work with a finite structure that abstracts the languages over Σ for effect annotations, over which we can compute fixed points.

Let $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$ be join-semilattices, that is, a partial order together with a join operation. We use the symbols $\sqsubseteq$, $\sqcup$ to refer to their generic structure of join-semilattices. We call $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ a *Büchi algebra* if it is equipped with the following operations:

- a binary operation on $\mathcal{M}_*$, written as $\mathcal{A} \cdot \mathcal{B}$ for $\mathcal{A}, \mathcal{B} \in \mathcal{M}_*$,
- a mapping $\mathcal{M}_* \times \mathcal{M}_{\leq\omega} \to \mathcal{M}_{\leq\omega}$, also written as $\mathcal{A} \cdot \mathcal{V}$ for $\mathcal{A} \in \mathcal{M}_*$ and $\mathcal{V} \in \mathcal{M}_{\leq\omega}$,
- an operation $(-)^\omega : \mathcal{M}_* \to \mathcal{M}_{\leq\omega}$,

such that all the operations are monotone and the binary operations are associative, *i.e.*, $\mathcal{A} \cdot (\mathcal{B} \cdot \mathcal{U}) = (\mathcal{A} \cdot \mathcal{B}) \cdot \mathcal{U}$ for any $\mathcal{A}, \mathcal{B} \in \mathcal{M}_*$ and $\mathcal{U} \in \mathcal{M}_* \cup \mathcal{M}_{\leq\omega}$.

For instance, $\mathcal{P}(\Sigma^*)$ and $\mathcal{P}(\Sigma^{\leq\omega})$ form a Büchi algebra. They are actually complete lattices. But note that meets are not required here because set intersections are not needed in the type system (see the typing rules in Figure 1). We are looking for a *finite* Büchi algebra $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ that abstracts them.

Recall that a *Galois insertion* between partial orders A, C consists of monotone maps $\alpha : C \to A$ and $\gamma : C \to A$ such that, for all $a \in A$ and $c \in C$,

$$\alpha(\gamma(a)) = a \qquad \gamma(\alpha(c)) \sqsupseteq c.$$

Intuitively, the function α furnishes for each concrete value in C its *abstraction*, whereas γ *concretizes* the abstract values in A. Moreover, the abstraction of c has a concretization lying above c.

Now we are ready to introduce the finite structure needed for presenting the type system and the inference algorithm. A *Büchi abstraction* is a *finite* Büchi algebra $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ equipped with Galois insertions (α_*, γ_*) between $\mathcal{M}_*$ and $\mathcal{P}(\Sigma^*)$ and $(\alpha_{\leq\omega}, \gamma_{\leq\omega})$ between $\mathcal{M}_{\leq\omega}$ and $\mathcal{P}(\Sigma^{\leq\omega})$ such that the abstraction functions α_* and $\alpha_{\leq\omega}$ are compatible with the algebraic structure, *i.e.*,

$$\alpha_*(A \cdot B) = \alpha_*(A) \cdot \alpha_*(B) \quad \alpha_{\leq\omega}(A \cdot V) = \alpha_*(A) \cdot \alpha_{\leq\omega}(V) \quad \alpha_{\leq\omega}(A^\omega) = \alpha_*(A)^\omega$$

for all $A, B \subseteq \Sigma^*$ and $V \subseteq \Sigma^{\leq\omega}$.

Readers may have expected more structures on a Büchi abstraction due to the analogy to $(\mathcal{P}(\Sigma^*), \mathcal{P}(\Sigma^{\leq\omega}))$. Some of them can be constructed. For instance, the least elements of $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$ are exactly the abstraction of the empty set, and thus we write $\bot = \alpha_{*/\leq\omega}(\emptyset)$. We can define finite iteration $(-)^*$ on $\mathcal{M}_*$ as the least fixed point by $\mathcal{A}^* = \mathsf{lfp}(\lambda X.\alpha_*(\{\varepsilon\}) \sqcup \mathcal{A} \cdot X)$, and then have $\alpha_*(A^\omega) = (\alpha_*(A))^\omega$. Moreover, we have the following properties of the Büchi abstraction.

Proposition 1. *1. The abstraction functions α_* and $\alpha_{\leq\omega}$ preserve joins,* i.e.*, $\alpha_*(A \cup B) = \alpha_*(A) \sqcup \alpha_*(B)$ and $\alpha_{\leq\omega}(U \cup V) = \alpha_{\leq\omega}(U) \sqcup \alpha_{\leq\omega}(V)$.*

2. *The concretization functions γ_* and $\gamma_{\leq\omega}$ may not preserve joins or concatenations, but they satisfy*

$$\gamma_*(\mathcal{A} \sqcup \mathcal{B}) \supseteq \gamma_*(\mathcal{A}) \cup \gamma_*(\mathcal{B}) \qquad \gamma_*(\mathcal{A} \cdot \mathcal{B}) \supseteq \gamma_*(\mathcal{A}) \cdot \gamma_*(\mathcal{B})$$
$$\gamma_{\leq\omega}(\mathcal{U} \sqcup \mathcal{V}) \supseteq \gamma_{\leq\omega}(\mathcal{U}) \cup \gamma_{\leq\omega}(\mathcal{V}) \qquad \gamma_{\leq\omega}(\mathcal{A} \cdot \mathcal{V}) \supseteq \gamma_*(\mathcal{A}) \cdot \gamma_{\leq\omega}(\mathcal{V}).$$

3. *Lastly, we have $\alpha_{\leq\omega}(\gamma_*(\mathcal{A})^\omega) = \mathcal{A}^\omega = \mathcal{A} \cdot \mathcal{A}^\omega$.*

Notice that, differing from finite iteration, here we do not define (or require) ω-iteration to be the greatest fixed point. We have only $\mathcal{A}^\omega \sqsubseteq \mathsf{gfp}(\lambda X.\mathcal{A} \cdot X)$ in general, because $\mathcal{A}^\omega$ is a fixed point of $\lambda X.\mathcal{A} \cdot X$ by Proposition 1(3). See Example 4 where a ω-iteration is strictly smaller than the greatest fixed point.

Example 4. Let the alphabet be $\Sigma = \{a\}$. We give a concrete example for a Büchi abstraction that distinguishes empty from non-empty words: $\mathcal{M}_* = \{\emptyset, \{\varepsilon\}, a^+, a^*\}$ and $\mathcal{M}_{\leq\omega} = \{\emptyset, \{\varepsilon\}, a^+, a^*, a^\omega, \{\varepsilon\} \cup a^\omega, a^+ \cup a^\omega, a^* \cup a^\omega\}$. For both, $\sqcup$ and $\sqsubseteq$ are set union and inclusion. All other operations are given by the evident operations on languages. The abstraction function α_* is determined by $\alpha_*(\{\varepsilon\}) = \{\varepsilon\}$ and $\alpha_*(X) = a^+$ for any $X \subseteq a^+$. All other values are determined by preservation of least element and joins. Similarly, $\alpha_{\leq\omega}$ is determined by $\alpha_{\leq\omega}(\{\varepsilon\}) = \{\varepsilon\}$, $\alpha_{\leq\omega}(X) = a^+$ for any $X \subseteq a^+$ and $\alpha_{\leq\omega}(a^\omega) = a^\omega$. The concretion functions γ_* and $\gamma_{\leq\omega}$ are both the identity.

We note that the greatest solution of the equation $X = a^+ \cdot X$ over $\mathcal{M}_{\leq\omega}$ is $a^+ \cup a^\omega \in \mathcal{M}_{\leq\omega}$, which can be seen by direct calculation. When calculated in $\mathcal{P}(\Sigma^{\leq\omega})$, the greatest solution is a^ω. □

As will become clear, a Büchi abstraction is sufficient for presenting the type system and the type inference algorithm. Hofmann and Chen [9,8] construct a Büchi abstraction from an *extended Büchi automaton* $\mathfrak{A}$ whose language is the union of the language of $\mathfrak{A}$ when understood as a traditional nondeterministic finite automaton and the language of $\mathfrak{A}$ when understood as a traditional Büchi automaton. A programming guideline is simply a set of finite and infinite traces, and thus can be represented as an extended Büchi automaton. Moreover, the Büchi abstraction of Hofmann and Chen is faithful with respect to acceptance by the automaton, which makes it suitable for representing effects in our type system. However, the development of the system (Section 5.2) and the type inference algorithm (Section 5.3) is *independent* of the construction of the Büchi abstraction. Readers should be able to access it without probing into the construction of Hofmann and Chen.

Similarly to the development in the previous section, we work with expressions in the system to capture non-terminating traces. These expressions take the form $\bigsqcup_{X \in \mathfrak{X}}(\mathcal{A}_X \cdot X) \sqcup \mathcal{B}$ where $\mathcal{A}_X \in \mathcal{M}_*$ and $\mathcal{B} \in \mathcal{M}_{\leq\omega}$. Let $\mathcal{V}(\mathfrak{X})$ range over such expressions. We define the operations of concatenation $\mathcal{A} \cdot \mathcal{V}(\mathfrak{X})$, join $\mathcal{V}(\mathfrak{X}) \sqcup \mathcal{V}'(\mathfrak{X})$ and instantiation $\mathcal{V}(\eta)$ in the same way as in Section 4. To relate to the non-algorithmic type system in Section 4, we extend the abstraction and concretization functions to expressions: Given $\mathcal{V}(\mathfrak{X}) = \bigsqcup_{X \in \mathfrak{X}}(\mathcal{A}_X \cdot X) \sqcup \mathcal{B}$, we concretize it by $\gamma(\mathcal{V}(\mathfrak{X})) = \bigcup_{X \in \mathfrak{X}}(\gamma_*(\mathcal{A}_X) \cdot X) \cup \gamma_{\leq\omega}(\mathcal{B})$. Similarly, we can

abstract trace expressions $V(\mathfrak{X})$, written as $\alpha(V(\mathfrak{X}))$. By the definition and properties of Büchi abstraction, we have:

$$\begin{aligned} \alpha(\gamma(\mathcal{V}(\mathfrak{X}))) &= \mathcal{V}(\mathfrak{X}) & \gamma(\alpha(V(\mathfrak{X}))) &\supseteq V(\mathfrak{X}) \\ \alpha(V(\mathfrak{X}) \cup V'(\mathfrak{X})) &= \alpha(V(\mathfrak{X})) \sqcup \alpha(V'(\mathfrak{X})) & \alpha(A \cdot V(\mathfrak{X})) &= \alpha_*(A) \cdot \alpha(V(\mathfrak{X})) \\ \gamma(\mathcal{V}(\mathfrak{X}) \sqcup \mathcal{V}'(\mathfrak{X})) &\supseteq \gamma(\mathcal{V}(\mathfrak{X})) \cup \gamma(\mathcal{V}'(\mathfrak{X})) & \gamma(\mathcal{A} \cdot \mathcal{V}(\mathfrak{X})) &= \gamma_*(\mathcal{A}) \cdot \gamma(\mathcal{V}(\mathfrak{X})). \end{aligned}$$

We lift these operations also to pairs of terminating effects and trace expressions by defining $\alpha(U, V(\mathfrak{X})) = (\alpha_*(U), \alpha(V(\mathfrak{X})))$ and $\gamma(\mathcal{U}, \mathcal{V}(\mathfrak{X})) = (\gamma_*(\mathcal{U}), \gamma(\mathcal{V}(\mathfrak{X})))$.

5.2 Büchi type and effect system

In this section, we assume a programming guideline given by the language $\mathfrak{A}$ and a Büchi abstraction $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ which is faithful w.r.t. $\mathfrak{A}$, *i.e.*, $\gamma_{\leq\omega}(\alpha_{\leq\omega}(\mathfrak{A})) = \mathfrak{A}$. Similarly to the previous development, we assume also a finite set *Reg* of regions and the sets $\mathit{Cls}(r)$ of classes that are allowed in a region. Moreover, we need to assume how to assign a region to a program location (see Appendix D).

We now present an algorithmic type system with effects being elements of $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$. The development is similar to the one in the previous section. To make the type system algorithmic, we remove the subtyping rule and replace sets of traces by their abstractions in the type system. Thanks to the finite abstraction, we can compute greatest solutions to capture the possible non-terminating traces of a program. However, the abstraction may not preserve greatest fixed points in general. To make the analysis more precise, we concretize the abstract expressions when defining well-typed programs, and later perform an algorithm to compute the non-terminating effects that abstract the greatest fixed point of the concretized expressions.

In a *Büchi class table* $(F, M_{\mathfrak{A}})$, the field typing $F : \mathit{Mtd} \times \mathit{ATyp} \rightharpoonup \mathit{Typ}$ is exactly the same as the one in the previous setting, but the method typing $M_{\mathfrak{A}} : \mathit{Mtd} \times \mathit{ATyp} \times \mathit{ATyp}^* \rightharpoonup \mathit{Typ} \times \mathcal{M}_* \times \mathcal{M}_{\leq\omega}$ gives the abstraction of the sets of possible traces for each method signature.

A *Büchi typing judgment* takes the form $\Gamma \vdash_{\mathfrak{A}} e : \tau \;\&\; (\mathcal{U}, \mathcal{V}(\mathfrak{X}))$ where $\mathcal{U} \in \mathcal{M}_*$ abstracts the set of traces generated during the terminating executions of e, and some instantiation of $\mathcal{V}(\mathfrak{X})$ abstracts the set of non-terminating traces. The typing rules are adapted correspondingly. For example, the Prim rule states that $\texttt{emit}(a)$ has effect $\alpha(\{a\}, \emptyset)$, that is, the pair of $\alpha_*(\{a\})$ and $\alpha_{\leq\omega}(\emptyset)$. The complete rules are given in Appendix D. As mentioned above, the system does not have the rule of subtyping.

A program $(\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ is *well-typed* w.r.t. $(F, M_{\mathfrak{A}})$ if there exists a family $\{\mathcal{V}_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M_{\mathfrak{A}})\}$ of expressions such that for every $\delta = (m, C_r, \bar{\sigma}) \in \mathrm{dom}(M_{\mathfrak{A}})$ with $C \in \mathit{Cls}(r)$ and $M_{\mathfrak{A}}(\delta) = (\tau, \mathcal{U}, \mathcal{V})$ and $\mathit{mtable}(C, m) = (\bar{x}, e)$, the following hold:

1. $\texttt{this} : C_r,\ \bar{x} : \bar{\sigma} \vdash_{\mathfrak{A}} e : \tau \;\&\; (\mathcal{U}, \mathcal{V}_\delta(\mathfrak{X}))$, and
2. $\eta_\gamma(X_\delta) \subseteq \gamma_{\leq\omega}(\mathcal{V})$,

where $\eta_\gamma : \mathfrak{X} \to \mathcal{P}(\Sigma^{\leq\omega})$ is the assignment function given by the greatest solution of the equation system $\{X_\delta = \gamma(\mathcal{V}_\delta(\mathfrak{X})) \mid \delta \in \mathrm{dom}(M_{\mathfrak{A}})\}$.

Remark 1. For the second condition, we use the greatest solution η_γ of the equation system $\{X_\delta = \gamma(\mathcal{V}_\delta(\mathfrak{X})) \mid \delta \in \mathrm{dom}(M_{\mathfrak{A}})\}$, where the expressions $\mathcal{V}_\delta(\mathfrak{X})$ are concretized, to characterize the possible traces generated by the non-terminating execution of the program. This seemingly loses the point of working with finitary abstraction. How are we going to compute the greatest fixed point? One may expect to compute the greatest solution $\eta : \mathfrak{X} \to \mathcal{M}_{\leq\omega}$ of the equation system $\{X_\delta = \mathcal{V}_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M_{\mathfrak{A}})\}$ without concretization and formulate the second condition as $\eta(X_\delta) \sqsubseteq \mathcal{V}$. Here, we work with the greatest solution η_γ of the concretized equations for two reasons: (1) The Büchi abstraction may not preserve greatest fixed points, which may lead to a loss in precision. For example, when we have just one method that calls itself, the equation system may have the form $\{X_\delta = \alpha_*(\{a\}) \cdot X_\delta\}$. If we solve it in the Büchi abstraction from Example 4, then we get the solution $\eta(X_\delta) = a^+ \cup a^\omega$, as noted there. However, if we concretize this equation and solve over sets, then we get the solution $\eta_\gamma(X_\delta) = a^\omega$. This means that for $\mathcal{V}$ we can choose a better approximation of the infinite traces if we solve over sets. The soundness theorem below shows that the more precise assignment η_γ already suffices to capture all the possible non-terminating traces. (2) We can compute over the Büchi abstraction a solution that abstracts the greatest solution η_γ of $\{X_\delta = \mathcal{V}_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M_{\mathfrak{A}})\}$. A simple algorithm to compute such a solution is given in Section 5.3.

Given a Büchi method typing $M_{\mathfrak{A}}$, we obtain a method typing $M^\gamma_{\mathfrak{A}}$ with concretized effects by defining for $\delta \in \mathrm{dom}(M_{\mathfrak{A}})$ with $M_{\mathfrak{A}}(\delta) = (\tau, \mathcal{U}, \mathcal{V})$:

$$M^\gamma_{\mathfrak{A}}(\delta) = (\tau, \gamma_*(\mathcal{U}), \gamma_{\leq\omega}(\mathcal{V})).$$

We relate the Büchi type system to the non-algorithmic type system introduced in Section 4 as follows.

Lemma 1. *If $\Gamma \vdash_{\mathfrak{A}} e : \tau \;\&\; (\mathcal{U}, \mathcal{V}(\mathfrak{X}))$ is derivable w.r.t. the Büchi class table $(F, M_{\mathfrak{A}})$, then so is $\Gamma \vdash e : \tau \;\&\; \gamma(\mathcal{U}, \mathcal{V}(\mathfrak{X}))$ w.r.t. the corresponding class table $(F, M^\gamma_{\mathfrak{A}})$.*

Proof. By induction on the derivation of $\Gamma \vdash_{\mathfrak{A}} e : \tau \;\&\; (\mathcal{U}, \mathcal{V}(\mathfrak{X}))$. □

The following is a direct consequence of Lemma 1.

Theorem 2. *If a program P is well-typed w.r.t. the Büchi class table $(F, M_{\mathfrak{A}})$, then it is well-typed w.r.t. the corresponding class table $(F, M^\gamma_{\mathfrak{A}})$.*

Therefore, the soundness of the Büchi type system follows directly from the soundness of the non-algorithmic type system (Theorem 1).

Corollary 1 (Soundness). *Let P be a well-typed program w.r.t. $(F, M_{\mathfrak{A}})$. For any m, C, r and $\bar\sigma$ with $M_{\mathfrak{A}}(m, C_r, \bar\sigma) = (\tau, \mathcal{U}, \mathcal{V})$ and for any x, $\bar y$, s, h, v, h' and w, we have*

1. $x : C_r, \bar{y} : \bar{\sigma} \vdash x.m(\bar{y}) : \tau \;\&\; (\mathcal{U}, X_{m,C_r,\bar{\sigma}})$;
2. *if* $(s,h) \vdash x.m(\bar{y}) \Downarrow v, h' \;\&\; w$ *then* $w \in \gamma_*(\mathcal{U})$;
3. *if* $(s,h) \vdash x.m(\bar{y}) \Uparrow \;\&\; w$, *then* $w \in \gamma_{\leq\omega}(\mathcal{V})$.

To certify that all the traces of a given method with signature δ are allowed by the language $\mathfrak{A}$, we only need to check if $\mathcal{U} \sqcup \mathcal{V} \sqsubseteq \alpha_{\leq\omega}(\mathfrak{A})$ where $M_{\mathfrak{A}}(\delta) = (_, \mathcal{U}, \mathcal{V})$, because the Büchi abstraction is faithful w.r.t. $\mathfrak{A}$, *i.e.*, $\gamma_{\leq\omega}(\alpha_{\leq\omega}(\mathfrak{A})) = \mathfrak{A}$. Note that $\mathcal{U} \in \mathcal{M}_*$ can be treated as an element of $\mathcal{M}_{\leq\omega}$, because there is an 'embedding' of $\mathcal{M}_*$ into $\mathcal{M}_{\leq\omega}$ given by $\alpha_{\leq\omega} \circ \gamma_*$ using the fact $\mathcal{P}(\Sigma^*) \subseteq \mathcal{P}(\Sigma^{\leq\omega})$. Therefore, the above join $\mathcal{U} \sqcup \mathcal{V}$ 'type-checks'.

5.3 Type inference algorithm

Given a program P that is well-typed w.r.t. a given FJ class table (F_0, M_0), one can construct a field typing $F : \mathit{Fld} \times \mathit{ATyp} \rightharpoonup \mathit{Typ}$ and a method typing $M^*_{\mathfrak{A}} : \mathit{Mtd} \times \mathit{ATyp} \times \mathit{ATyp}^* \rightharpoonup \mathit{Typ} \times \mathcal{M}_*$ without non-terminating effects using the inference algorithm of [5, Appendix F] with some adaptions for the Büchi abstraction. Here, we extend their algorithm to calculate also non-terminating effects to get a Büchi class table $(F, M_{\mathfrak{A}})$ with respect to which P is well-typed.

Our Büchi effect type system is algorithmic, *i.e.*, it prescribes an algorithm for building typing derivations. We follow the typing rules to construct a procedure typeff that maps an environment Γ and a term e to a type τ, a finitary effect $\mathcal{U}$ and an expression $\mathcal{V}(\mathfrak{X})$, if e is typable in Γ, such that $\Gamma \vdash e : \tau \;\&\; (\mathcal{U}, \mathcal{V}(\mathfrak{X}))$ is derivable. In particular, typeff may update the table F, which happens in the case of a field-set statement $x.f = y$. The SET rule requires that y must have a subtype of the types allowed for the field f by F. If this condition is not satisfied, then typeff updates F by joining the offending entry with the type of y. We make use typeff to construct a Büchi class table as follows.

In brief, both terminating and non-terminating effects are computed as fixed points using the typeff procedure. We firstly construct a table $(F, M^{\mathfrak{X}}_{\mathfrak{A}})$ where $M^{\mathfrak{X}}_{\mathfrak{A}}$ assigns to each method signature a type, a terminating effect and a trace expression. We start with such a table whose entries are the possibly "lowest" types and effects, by lifting (F_0, M_0) with empty regions and effects. For each method signature, we "increase" its type and effects in the table if they are different from the result given by calling typeff with its method body. We repeat this until the table cannot be updated further. The resulting table $(F, M^{\mathfrak{X}}_{\mathfrak{A}})$ encodes the types, the terminating effects and the traces expressions of the program. By applying a simple algorithm, we compute a solution of the equation system formed by the trace expressions. This solution captures all the possible non-terminating traces of the program. The detailed inference algorithm is presented as follows.

Step 1: Lift (F_0, M_0) to a table $(F, M^{\mathfrak{X}}_{\mathfrak{A}})$ with empty regions and effects. For the field typing F, we define $F(C_r, f) = D_\emptyset$ for each field $f \in \mathit{fields}(C)$ with $F_0(C, f) = D$ and for each region $r \in \mathit{Reg}$. For the method typing $M^{\mathfrak{X}}_{\mathfrak{A}}$, we define $M^{\mathfrak{X}}_{\mathfrak{A}}(C_r, m, ((D_1)_{s_1}, \ldots, (D_n)_{s_n})) = (E_\emptyset, \alpha(\emptyset, \emptyset))$ for each method $m \in \mathit{methods}(C)$ with $M_0(C, m) = (\bar{D}, E)$ and for each region $r \in \mathit{Reg}$, where α is the abstraction function for pairs of terminating effects and trace expressions.

Step 2: Update $(F, M^{\mathfrak{X}}_{\mathfrak{A}})$. For each method signature, we use the typeff procedure to compute the type and effects of its body. If they are different from those encoded in the current table, then update the corresponding entry in $M^{\mathfrak{X}}_{\mathfrak{A}}$. Specifically, for each $\delta = (m, C_r, \bar{\sigma}) \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})$ with $mtable(C, m) = (\bar{x}, e)$ and $C \in Cls(r)$, we call typeff with the environment $\texttt{this} : C_r$, $\bar{x} : \bar{\sigma}$ and the term e, and obtain a type τ, a terminating effect $\mathcal{U}$ and a trace expression $\mathcal{V}(\mathfrak{X})$. Then we set $M^{\mathfrak{X}}_{\mathfrak{A}}(\delta) = M^{\mathfrak{X}}_{\mathfrak{A}}(\delta) \sqcup (\tau, \mathcal{U}, \mathcal{V}(\mathfrak{X}))$, where joins are constructed componentwise. Note that the table F may also be updated if e contains field-set sub-expressions.

Step 3: Ensure that F and $M^{\mathfrak{X}}_{\mathfrak{A}}$ are well-formed. For instance, if $C \preceq D$ and $F(C_r, f) \neq F(D_r, f)$ for some field f and region r, then both of them are set to $F(C_r, f) \sqcup F(D_r, f)$.

Step 4: Repeat Steps 2 and 3 until there is no further update for $(F, M^{\mathfrak{X}}_{\mathfrak{A}})$. Notice that in each iteration the updated entries in F and $M^{\mathfrak{X}}_{\mathfrak{A}}$ get "worsened", *i.e.*, the types or effects increase. Because there are only finitely many types and (abstract) effects, we will reach the fixed point after some iterations.

The resulting table $M^{\mathfrak{X}}_{\mathfrak{A}}$ encodes the types, the terminating effects and the associated family $\{\mathcal{V}_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})\}$ of expressions of the program, *i.e.*, for each $\delta = (m, C_r, \bar{\sigma}) \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})$ with $M^{\mathfrak{X}}_{\mathfrak{A}}(\delta) = (\tau, \mathcal{U}, \mathcal{V}(\mathfrak{X}))$ and $mtable(C, m) = (\bar{x}, e)$ and $C \in Cls(r)$, we can derive $\texttt{this} : C_r$, $\bar{x} : \bar{\sigma} \vdash_{\mathfrak{A}} e : \tau \;\&\; (\mathcal{U}, \mathcal{V}_\delta(\mathfrak{X}))$.

Step 5: Solve the equation system $\{X_\delta = \mathcal{V}_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})\}$. We firstly rewrite each expression $\mathcal{V}_\delta(\mathfrak{X})$ to an equivalent form of $\mathcal{A}_\delta \cdot X_\delta \sqcup \mathcal{W}_\delta(\mathfrak{X} - X_\delta)$. Then we iterate the following steps for each $\delta \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})$:

i) Set $X_\delta = \mathcal{A}^*_\delta \cdot \mathcal{W}_\delta(\mathfrak{X} - X_\delta) \sqcup \mathcal{A}^\omega_\delta$.
ii) Substitute X_δ by $\mathcal{A}^*_\delta \cdot \mathcal{W}_\delta(\mathfrak{X} - X_\delta) \sqcup \mathcal{A}^\omega_\delta$ on the right hand side of all the other equations.

Because one variable is eliminated on the right hand side of the equations after each iteration, in the end we solve the system and obtain an assignment function $\eta : \mathfrak{X} \to \mathcal{M}_{\leq\omega}$. Notice that η may not be the greatest solution but it abstracts the greatest solution η_γ of $\{X_\delta = \gamma(\mathcal{V}(\mathfrak{X})) \mid \delta \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})\}$ of concretized equations. For instance, let us consider a singleton equation system $\{X = \mathcal{A} \cdot X \sqcup \mathcal{B}\}$. Using the above algorithm, we get a solution $\eta(X) = \mathcal{A}^* \cdot \mathcal{B} \sqcup \mathcal{A}^\omega$. The greatest solution of $\{X = \gamma_*(\mathcal{A}) \cdot X \cup \gamma_{\leq\omega}(B)\}$ is $\eta_\gamma(X) = \gamma_*(\mathcal{A})^* \cdot \gamma_{\leq\omega}(\mathcal{B}) \cup \gamma_*(\mathcal{A})^\omega$. It is not hard to prove $\eta(X) = \alpha_{\leq\omega}(\eta_\gamma(X))$ using Proposition 1. One can easily generalize the proof to systems with multiple equations. We refer the reader to *e.g.* [6] for the characterization of simultaneous fixed points.

Step 6: Convert $M^{\mathfrak{X}}_{\mathfrak{A}}$ to a method typing $M_{\mathfrak{A}}$ with non-terminating effects. We simply define $M_{\mathfrak{A}}(\delta) = (\tau, \mathcal{U}, \eta(X_\delta))$ for every $\delta = (m, C_r, \bar{\sigma}) \in \mathrm{dom}(M^{\mathfrak{X}}_{\mathfrak{A}})$ with $M^{\mathfrak{X}}_{\mathfrak{A}}(\delta) = (\tau, \mathcal{U}, \mathcal{V}(\mathfrak{X}))$.

Theorem 3. *Let P be a program that is well-typed w.r.t. some standard class table (F_0, M_0). Then the above algorithm gives a Büchi class table $(F, M_{\mathfrak{A}})$ with respect to which P is well-typed.*

6 Conclusion

This paper presents a framework to verify if a given Featherweight Java program adheres to a given programming guideline. Let $\mathfrak{A}$ be a language representing the guideline. Suppose a Büchi abstraction that is faithful w.r.t. $\mathfrak{A}$ is given, *e.g.* the one of Hofmann and Chen [9]. From the standard Java types of the fields and methods, we construct a Büchi class table $(F, M_{\mathfrak{A}})$ with respect to which the program is well-typed. In addition to the return type, the table $M_{\mathfrak{A}}$ provides the terminating and non-terminating effects to each method signature. If the effects of a method are included in the abstraction of $\mathfrak{A}$, then any event trace generated by the method is allowed by $\mathfrak{A}$ and thus the method adheres to the guideline.

While the general idea of a Java type system with Büchi effects was already sketched in [8], its meta-theoretic properties like soundness and the correctness of type inference were not investigated (in fact, the proposed notion of well-typedness in [8] is unclear w.r.t. the quantification of η) and some proposed choices were a little limited (such as the choice of the variables $\mathfrak{X}$ with one variable per region rather than one per method signature). Here we develop this idea fully and establish the meta-theory. Our definition of well-typedness clarifies the role of greatest fixed points. It enables a simple soundness proof by a single induction on approximations of a greatest fixed point, without reference to the particular solution algorithm used in type inference. This should make it easier to consider alternative algorithms, such as worklist algorithms [12]. Furthermore, we make the type system independent from the automata-theoretic constructions of [8] by capturing the required structure abstractly in terms of a Büchi abstraction.

Compared to the region typing system of [5], we improve the precision of region typing by information about the actual classes of objects in a region available for typing. We have illustrated in Section 4 that regions allow for static class analysis, which is well-known to be useful for object oriented programs, see *e.g.* [3]. We have omitted two features from the type system of [5] that are orthogonal to the concerns in this paper and that can be added easily. First, we have not included the taintedness analysis for strings from [5]. This can be integrated in our type system in exactly the same way. It does not require any change to our extension of the type system for non-terminating effects, because there are no diverging operational semantics rules for strings. Second, the type system in [5] is context-sensitive. The typing judgement is parametrized with calling contexts, which can be considered as abstractions of Java's stack traces, in order to improve the precision of analysis (see [1] for a detailed discussion). The idea is that each method may be given a different type depending on its calling context. Hence, method typing depends also on contexts. Correspondingly, our assignment to trace variables should also depend on contexts. But the development of the soundness proof and the type inference algorithm is almost the same. We have omitted it to focus on the essential aspects of our work.

We have motivated the type system as a tool to analyze simple program properties. We have a prototype implementation of type inference based on Soot [13] that has verified the examples in this paper. We are in the process of developing it into a full analysis tool.

References

1. Beringer, L., Grabowski, R., Hofmann, M.: Verifying pointer and string analyses with region type systems. Computer Languages, Systems & Structures **39**(2), 49–65 (2013). https://doi.org/10.1016/j.cl.2013.01.001
2. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Graham, R.M., Harrison, M.A., Sethi, R. (eds.) Principles of Programming Languages (POPL 1977). pp. 238–252 (1977). https://doi.org/10.1145/512950.512973
3. Dean, J., Grove, D., Chambers, C.: Optimization of object-oriented programs using static class hierarchy analysis. In: Olthoff, W.G. (ed.) ECOOP'95 - European Conference on Object-Oriented Programming (ECOOP 1995). Lecture Notes in Computer Science, vol. 952, pp. 77–101. Springer (1995)
4. Distefano, D., Fähndrich, M., Logozzo, F., O'Hearn, P.W.: Scaling static analyses at Facebook. Communications of the ACM **62**(8), 62–70 (2019). https://doi.org/10.1145/3338112
5. Erbatur, S., Hofmann, M., Zălinescu, E.: Enforcing programming guidelines with region types and effects. In: Chang, B.Y.E. (ed.) Programming Languages and Systems (APLAS 2017). Lecture Notes in Computer Science, vol. 10695, pp. 85–104. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-71237-6_5
6. Fritz, C.: Some fixed point basics. In: Grädel, E., Thomas, W., Wilke, T. (eds.) Automata Logics, and Infinite Games, Lecture Notes in Computer Science, vol. 2500, pp. 359–364. Springer, Berlin, Heidelberg (2002). https://doi.org/10.1007/3-540-36387-4_20
7. Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A.: The Java Language Specification. Oracle America, Inc., Java SE 14 edn. (2020), `http://docs.oracle.com/javase/specs/jls/se14/jls14.pdf`
8. Hofmann, M., Chen, W.: Abstract interpretation from Büchi automata. In: Computer Science Logic and Logic in Computer Science (CSL-LICS 2014). pp. 51:1–51:10. Association for Computing Machinery (2014). https://doi.org/10.1145/2603088.2603127
9. Hofmann, M., Chen, W.: Büchi types for infinite traces and liveness (2014), technical report, `arXiv:1401.5107 [cs.LO]`
10. Igarashi, A., Pierce, B.C., Wadler, P.: Featherweight Java: a minimal core calculus for Java and GJ. ACM Transactions on Programming Languages and Systems **23**(3), 396–450 (2001). https://doi.org/10.1145/503502.503505
11. Khoury, R., Tawbi, N.: Which security policies are enforceable by runtime monitors? A survey. Computer Science Review **6**(1), 27–45 (2012). https://doi.org/10.1016/j.cosrev.2012.01.001
12. Nielson, F., Nielson, H.R., Hankin, C.: Principles of program analysis (2. corr. print). Springer (2005). https://doi.org/10.1007/978-3-662-03811-6
13. Sable Group, M.U.: Soot - A framework for analyzing and transforming Java and Android Applications, URL: `https://soot-oss.github.io/soot/`

A An additional example

The following additional example illustrates the interaction of regions and subtyping. It supplements the example from the main text, where we have focused on infinitary effects.

Example 5. Consider a program with two simple classes (the return type is `Object` only because we do not have a `void` type):

```
class A {
  Object f() {
    emit(a);
  }
}
```

```
class B extends A {
  Object f() {
    emit(b);
  }
}
```

Suppose r_0, r_1 and r_2 are regions with $\mathit{Cls}(r_0) = \mathit{Cls}$, $\mathit{Cls}(r_1) = \{A\}$ and $\mathit{Cls}(r_2) = \{B\}$. Then the program is well-typed with respect to the following typing table:

```
class A_{r0} {
  Object f() & ({a,b},∅);
}
```

```
class B_{r0} {
  Object f() & ({b},∅);
}
```

```
class A_{r1} {
  Object f() & ({a},∅);
}
```

```
class B_{r1} {
  Object f() & (∅,∅);
}
```

```
class A_{r2} {
  Object f() & ({b},∅);
}
```

```
class B_{r2} {
  Object f() & ({b},∅);
}
```

The types in region r_0 should be unsurprising. The well-formedness condition on method tables requires us to include the effect b also in class A_{r_0}. In regions r_1 and r_2 we can ignore the bodies of `A.f` and `B.f` respectively when constructing the method table. This is because the definition of well-typedness for programs makes a requirement only for methods entries $\delta = (m, C_r, \overline{\sigma})$ with $C \in \mathit{Cls}(r)$. Here we have $B \notin \mathit{Cls}(r_1)$ and $A \notin \mathit{Cls}(r_2)$, so we can ignore the bodies of `A.f` and `B.f` in regions r_1 and r_2 respectively.

With the above typing table, consider the following code fragment.

```
A x = new B();
x.f();
```

The type system allows us to give `x` type A_{r_2}. In this case, the method call `x.f()` is analysed to have effect $\{b\}$, which is exact. We could also give `x` the type A_{r_0}, but this choice would lead to an over-approximation of the actual effects. We would get that the method call `x.f()` has effect $\{a, b\}$. We would not be allowed to give `x` the type A_{r_1}, since the premise of NEW would require B to be in $\mathit{Cls}(r_1)$. □

B Operational semantics of Featherweight Java

The operational semantics $(s,h) \vdash e \Downarrow v, h' \,\&\, w$ of the terminating evaluation of e is defined inductively from the following inference rules:

$$\frac{}{(s,h) \vdash \texttt{emit}(a) \Downarrow \mathit{null}, h \,\&\, a}$$

$$\frac{}{(s,h) \vdash x \Downarrow s(x), h \,\&\, \varepsilon} \qquad \frac{}{(s,h) \vdash \texttt{null} \Downarrow \mathit{null}, h \,\&\, \varepsilon}$$

$$\frac{s(x) = s(y) \qquad (s,h) \vdash e_1 \Downarrow v, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Downarrow v, h' \,\&\, w}$$

$$\frac{s(x) \neq s(y) \qquad (s,h) \vdash e_2 \Downarrow v, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Downarrow v, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1 \qquad (s[x \mapsto v_1], h_1) \vdash e_2 \Downarrow v_2, h_2 \,\&\, w_2}{(s,h) \vdash \texttt{let } x = e_1 \texttt{ in } e_2 \Downarrow v_2, h_2 \,\&\, w_1 w_2}$$

$$\frac{l \notin \mathrm{dom}(h) \qquad F = [f \mapsto \mathit{null}]_{f \in \mathit{fields}(C)}}{(s,h) \vdash \texttt{new } C \Downarrow l, h[l \mapsto (C,F)] \,\&\, \varepsilon}$$

$$\frac{(s,h) \vdash e \Downarrow \mathit{null}, h' \,\&\, w}{(s,h) \vdash (C)\, e \Downarrow \mathit{null}, h' \,\&\, w} \qquad \frac{(s,h) \vdash e \Downarrow (D,_), h' \,\&\, w \qquad D \prec C}{(s,h) \vdash (C)\, e \Downarrow (D,_), h' \,\&\, w}$$

$$\frac{s(x) = l \qquad h(l) = (_, F)}{(s,h) \vdash x.f \Downarrow F(f), h \,\&\, \varepsilon}$$

$$\frac{s(x) = l \qquad h(l) = (C,F) \qquad h' = h[l \mapsto (C, F[f \mapsto s(y)])]}{(s,h) \vdash x.f := y \Downarrow s(y), h' \,\&\, \varepsilon}$$

$$\frac{\begin{array}{c} s(x) = l \qquad h(l) = (C,_) \qquad \mathit{mtable}(C,m) = (\bar{x}, e) \\ ([\texttt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i \in \{1,\ldots,|\bar{x}|\}}, h) \vdash e \Downarrow v, h' \,\&\, w \end{array}}{(s,h) \vdash x.m(\bar{y}) \Downarrow v, h' \,\&\, w}$$

And the operational semantics of divergence $(s,h) \vdash e \Uparrow \,\&\, w$ is interpreted coinductively from the following inference rules:

$$\frac{s(x) = s(y) \qquad (s,h) \vdash e_1 \Uparrow \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Uparrow \,\&\, w}$$

$$\frac{s(x) \neq s(y) \qquad (s,h) \vdash e_2 \Uparrow \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Uparrow \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1 \qquad (s[x \mapsto v_1], h_1) \vdash e_2 \Uparrow \,\&\, w_2}{(s,h) \vdash \texttt{let } x = e_1 \texttt{ in } e_2 \Uparrow \,\&\, w_1 w_2}$$

$$\frac{(s,h) \vdash e_1 \Uparrow \& \; w}{(s,h) \vdash \texttt{let}\; x = e_1 \;\texttt{in}\; e_2 \Uparrow \& \; w}$$

$$\frac{(s,h) \vdash e \Uparrow \& \; w}{(s,h) \vdash (C)\, e \Uparrow \& \; w}$$

$$\frac{s(x) = l \qquad h(l) = (C, _) \qquad \mathit{mtable}(C,m) = (\bar{x}, e) \qquad ([\texttt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i \in \{1,\ldots,|\bar{x}|\}}, h) \vdash e \Uparrow \& \; w}{(s,h) \vdash x.m(\bar{y}) \Uparrow \& \; w}$$

C Soundness of the non-algorithmic type system

We follow [5, Appendix E.2] to interpret the typing judgment using heap typings.

Recall that a *heap typing* $\Pi : \mathit{Loc} \rightharpoonup \mathit{ATyp}$ assigns to each heap location a static class (an upper bound of the actual class found at that location) and a region for objects. A heap typing Π' *extends* a heap typing Π, written $\Pi' \sqsupseteq \Pi$, if $\mathrm{dom}(\Pi) \subseteq \mathrm{dom}(\Pi')$ and $\Pi(l) = \Pi'(l)$ for all $l \in \mathrm{dom}(\Pi)$.

A typing judgment for values takes the form $\Pi \vdash v : \tau$, meaning that the value v may be typed with τ according to the heap typing Π. Here are the typing rules for values:

$$\frac{}{\Pi \vdash \mathit{null} : \texttt{NullType}_\emptyset} \qquad \frac{\Pi(l) = C_r \qquad C \in \mathit{Cls}(r)}{\Pi \vdash l : C_r} \qquad \frac{\Pi \vdash v : \sigma \qquad \sigma \leq: \tau}{\Pi \vdash v : \tau}$$

We then lift it to stores and variable contexts: $\Pi \vdash s : \Gamma$ iff $\Pi \vdash s(x) : \Gamma(x)$ for all $x \in \mathrm{dom}(\Gamma)$.

Lastly, we say a heap h is *well-typed* w.r.t. a heap typing Π (and implicitly a class table (F, M)), written $\Pi \vdash h$, if $l \in \mathrm{dom}(h)$ and $\Pi \vdash h(l) : \Pi(l)$ for all $l \in \mathrm{dom}(\Pi)$, where $\Pi \vdash (C, G) : D_r$ iff $C = D$, $\mathrm{dom}(G) = \mathit{fields}(C)$ and $\Pi \vdash G(f) : F(f, C_r)$ for all $f \in \mathit{fields}(C)$. Roughly speaking, an object is well-typed if the value of each field is well-typed, and a heap is well-typed if the object at each location is well-typed.

Theorem 4 (Soundness). *Let P be a well-typed program w.r.t. the class table (F, M). Let $\eta : \mathfrak{X} \to \mathcal{P}(\Sigma^{\leq \omega})$ be the greatest solution of the associated expressions $\{V_\delta(\mathfrak{X}) \mid \delta \in \mathrm{dom}(M)\}$ of P. For any Γ, e, τ, U, $V(\mathfrak{X})$, Π, s, h, v, h' and w with*

$$\Gamma \vdash e : \tau \;\&\; (U, V(\mathfrak{X})) \quad \textit{and} \quad \Pi \vdash s : \Gamma \quad \textit{and} \quad \Pi \vdash h,$$

(1) if $(s,h) \vdash e \Downarrow v, h' \;\&\; w$, then there exists some $\Pi' \sqsupseteq \Pi$ such that

$$\Pi' \vdash v : \tau \quad \textit{and} \quad \Pi' \vdash h' \quad \textit{and} \quad w \in U;$$

(2) if $(s,h) \vdash e \Uparrow \& \; w$, then $w \in V(\eta)$.

Proof (of Theorem 4(1)). Similarly to the proof of [5, Theorem 5], we carry out the proof by induction over the sum of the depth of the derivation of the typing judgment and the depth of the derivation of the operational semantics judgment.

For the proof of Theorem 4(2), we use the following approximations of the greatest solution: Given a well-typed program w.r.t. (F, M) with associated expressions $\{A_\delta \cdot X_\delta \cup V_\delta(\mathfrak{X} - X_\delta) \mid \delta \in \mathrm{dom}(M)\}$, we define for each $n \in \mathbb{N}$ an assignment function $\eta_n : \mathfrak{X} \to \mathcal{P}(\Sigma^{\leq\omega})$ inductively by

$$\eta_0(X_\delta) = \Sigma^{\leq\omega} \qquad \eta_{n+1}(X_\delta) = A_\delta \cdot \eta_n(X_\delta) \cup V_\delta(\eta_n).$$

Let η be the greatest solution of $\{X_\delta = A_\delta \cdot X_\delta \cup V_\delta(\mathfrak{X} - X_\delta) \mid \delta \in \mathrm{dom}(M)\}$. Then we have $\eta(X_\delta) = \bigcap_{i\in\mathbb{N}} \eta_i(X_\delta)$ for every $\delta \in \mathrm{dom}(M)$. More generally, we have $\eta(X_\delta) = \bigcap_{i\in N} \eta_i(X_\delta)$ for any infinite subset $N \subseteq \mathbb{N}$.

We firstly show that the above approximations η_n satisfy the soundness theorem in the following sense:

Lemma 2. *Let P be a well-typed program w.r.t. (F, M). Let η_n's be the approximations to the greatest solution of P defined as above. For any n, Γ, e, τ, U, $V(\mathfrak{X})$, Π, s, h and w, if*

$$\Gamma \vdash e : \tau \,\&\, (U, V(\mathfrak{X})) \quad \textit{and} \quad \Pi \vdash s : \Gamma \quad \textit{and} \quad \Pi \vdash h \quad \textit{and} \quad (s, h) \vdash e \Uparrow \,\&\, w$$

then $w \in V(\eta_n)$.

Proof. By induction on n and on the depth of the typing derivation.

When $n = 0$, it is trivial because η_0 assigns $\Sigma^{\leq\omega}$ to all variables.

Assume that the statement holds for n. To show that it holds for $n + 1$, we perform an induction on the depth of the typing derivation.

If $\Gamma \vdash e : \tau \,\&\, (U, V(\mathfrak{X}))$ is derived by the subtyping rule, by rule inversion, we get $\Gamma \vdash e : \tau' \,\&\, (U', V'(\mathfrak{X}))$ with $\tau' \leq: \tau$, $U' \subseteq U$ and $V'(\mathfrak{X}) \subseteq V(\mathfrak{X})$. If $(s, h) \vdash e \Uparrow \,\&\, w$, by induction hypothesis, we have $w \in V'(\eta_{n+1}) \subseteq V(\eta_{n+1})$.

If the subtyping rule is not the last used rule of the typing derivation, we perform a case distinction over the last rule used in the derivation of operational semantics judgment. Then the cases of x, $\texttt{emit}(a)$, *null* and $\texttt{new}\, C$ are ruled out because their execution always terminates. Here we check the following cases.

- $\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2 \,\&\, (U_1 \cdot U_2, V_1(\mathfrak{X}) \cup U_1 \cdot V_2(\mathfrak{X}))$
  We have $\Gamma \vdash e_1 : \tau_1 \,\&\, (U_1, V_1(\mathfrak{X}))$ and $\Gamma, x : \tau_1 \vdash e_2 : \tau_2 \,\&\, (U_2, V_2(\mathfrak{X}))$ by rule inversion.
  The operational semantics judgment is $(s, h) \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \Uparrow \,\&\, w$. By rule inversion, we have two possibilities:
  - If $(s, h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1$ and $(s[x \mapsto v_1], h_1) \vdash e_2 \Uparrow \,\&\, w_2$, then we have $w = w_1 w_2$. By Theorem 4(1), we have $w_1 \in U_1$ and get a $\Pi' \sqsupseteq \Pi$ such that $\Pi' \vdash v_1 : \tau_1$ and $\Pi' \vdash h_1$, and thus, $\Pi' \vdash s[x \mapsto v_1] : (\Gamma, x : \tau_1)$. By induction hypothesis, we have $w_2 \in V_2(\eta_{n+1})$. Therefore, $w = w_1 w_2 \in U_1 \cdot V_2(\eta_{n+1}) \subseteq V_1(\eta_{n+1}) \cup U_1 \cdot V_2(\eta_{n+1})$.
  - If $(s, h) \vdash e_1 \Uparrow \,\&\, w$, then $w \in V_1(\eta_{n+1}) \subseteq V_1(\eta_{n+1}) \cup U_1 \cdot V_2(\eta_{n+1})$ by induction hypothesis.

– $\Gamma, x : C_R, \bar{y} : \bar{\sigma} \vdash x.m(\bar{y}) : \tau \;\&\; (U, \bigcup_{r\in R} X_{m,C_r,\bar{\sigma}})$
We have $(\tau, U) = \bigsqcup\{(\tau_r, U_r) \mid (\tau_r, U_r, _) = M(m, C_r, \bar{\sigma}), r \in R\}$ by rule inversion.
Suppose $\Pi \vdash s : (\Gamma, x : C_R, \bar{y} : \bar{\sigma})$ and $\Pi \vdash h$ and $(s, h) \vdash x.m(\bar{y}) \Uparrow \;\&\; w$.
The goal is to show $w \in \bigcup_{r\in R} \eta_{n+1}(X_{m,C_r,\bar{\sigma}})$.
Let $s(x) = l$, $h(l) = (D, _)$ and $mtable(C, m) = (\bar{x}, e)$. By rule inversion, we have $(s', h) \vdash e \Uparrow \;\&\; w$ where $s' = [\texttt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i\in\{1,\ldots,|\bar{x}|\}}$.
Let $\Pi(l) = E_r$. Because $\Pi \vdash s : (\Gamma, x : C_R, \bar{y} : \bar{\sigma})$, we have $\Pi \vdash l : C_R$ and thus $r \in R$ and $E \preceq C$. As $h(l) = (D, _)$, we have $E = D$ and thus $D \preceq C$.
Let $\Gamma' = (\texttt{this} : D_r, \bar{x} : \bar{\sigma})$. We then have $\Pi \vdash s' : \Gamma'$.
Let $\delta = (m, D_r, \bar{\sigma})$. Because the program is well-typed, we obtain a derivation of $\Gamma' \vdash e : \tau' \;\&\; (U', A_\delta \cdot X_\delta \cup V_\delta(\mathfrak{X} - X_\delta))$ where $(\tau', U', _) = M(\delta)$ and $A_\delta \cdot X_\delta \cup V_\delta(\mathfrak{X} - X_\delta)$ the associated expression of the program indexed by δ.
We then have $w \in A_\delta \cdot \eta_n(X_\delta) \cup V_\delta(\eta_n) = \eta_{n+1}(X_\delta) \subseteq \bigcup_{r\in R} \eta_{n+1}(X_{m,C_r,\bar{\sigma}})$ by induction hypothesis.

We leave the remaining cases to the reader.

Lemma 3. *For any expression $V(\mathfrak{X})$ and for any infinite set $N \subseteq \mathbb{N}$, we have $\bigcap_{i\in N} V(\eta_i) \subseteq V(\eta)$.*

Proof. By induction on the number of variables. The base case is trivial. For the inductive step, suppose an expression $A \cdot X \cup V(\mathfrak{X})$ with $X \notin \mathfrak{X}$ is given. We assume $w \in A \cdot \eta_i(X) \cup V(\eta_i)$ for all $i \in N$. Reasoning classically, there are two possible cases: (1) If $w \in A \cdot \eta_i(X)$ for all $i \in N'$ for some infinite $N' \subseteq N$, then we have $w \in A \cdot \eta(X)$ because η is the greatest fixed point. (2) If $w \in V(\eta_i)$ for all $i \in N'$ for some infinite $N' \subseteq N$, then $w \in V(\eta)$ by induction hypothesis.

The second half of the soundness theorem follows from the above lemmas:

Proof (of Theorem 4(2)). If $\Gamma \vdash e : \tau \;\&\; (U, V(\mathfrak{X}))$, $\Pi \vdash s : \Gamma$, $\Pi \vdash h$ and $(s, h) \vdash e \Uparrow \;\&\; w$, then, by the above lemmas, we have $w \in \bigcap_{n\in\mathbb{N}} V(\eta_n) \subseteq V(\eta)$.

D Büchi type and effect system

We annotate each expression e with an *expression label* $i \in Pos$ and write $[e]^i$ for Featherweight Java expressions. One can think of these labels as the line numbers in a Java program. In the paper, we consider only programs where each expression label appears at most once. We simply write e rather than $[e]^i$ when i is not important. In the Büchi type system, we assign a region to a new object based on its allocation site, that is, the label of the corresponding `new` expression. For this, we assume a given function $\psi : Pos \to Reg$ such that for any expression $[\texttt{new}\ C]^i$ if $\psi(i) = r$ then $C \in Cls(r)$. In words, the class of a new object is always allowed in the region assigned to the object.

Here are the rules of the Büchi type system introduced in Section 5. Here α is the abstraction function on pairs of terminating effects and trace expressions defined componentwise, *i.e.*, $\alpha(\mathcal{U}, \mathcal{V}(\mathfrak{X})) = (\alpha_*(\mathcal{U}), \alpha(\mathcal{V}(\mathfrak{X})))$.

$$\text{PRIM}\ \frac{}{\Gamma \vdash_{\mathfrak{A}} \texttt{emit}(a) : \texttt{NullType}_\emptyset \;\&\; \alpha(\{a\}, \emptyset)}$$

$$\textsc{var}\ \frac{}{\Gamma,\, x : \tau \vdash_{\mathfrak{A}} x : \tau \,\&\, \alpha(\{\varepsilon\}, \emptyset)}$$

$$\textsc{if}\ \frac{\begin{array}{c}\Gamma,\, x : C_{R\cap S},\, y : D_{R\cap S} \vdash_{\mathfrak{A}} e_1 : \tau_1 \,\&\, (\mathcal{U}_1, \mathcal{V}_1(\mathfrak{X})) \\ \Gamma,\, x : C_R,\ y : D_S \vdash_{\mathfrak{A}} e_2 : \tau_2 \,\&\, (\mathcal{U}_2, \mathcal{V}_2(\mathfrak{X}))\end{array}}{\Gamma,\, x : C_R,\, y : D_S \vdash_{\mathfrak{A}} \texttt{if}\ x = y\ \texttt{then}\ e_1\ \texttt{else}\ e_2 : \tau_1 \sqcup \tau_2 \,\&\, (\mathcal{U}_1 \sqcup \mathcal{U}_2, \mathcal{V}_1(\mathfrak{X}) \sqcup \mathcal{V}_2(\mathfrak{X}))}$$

$$\textsc{let}\ \frac{\Gamma \vdash_{\mathfrak{A}} e_1 : \tau_1 \,\&\, (\mathcal{U}_1, \mathcal{V}_1(\mathfrak{X})) \qquad \Gamma,\, x : \tau_1 \vdash_{\mathfrak{A}} e_2 : \tau_2 \,\&\, (\mathcal{U}_2, \mathcal{V}_2(\mathfrak{X}))}{\Gamma \vdash_{\mathfrak{A}} \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2 \,\&\, (\mathcal{U}_1 \cdot \mathcal{U}_2, \mathcal{V}_1(\mathfrak{X}) \sqcup \mathcal{U}_1 \cdot \mathcal{V}_2(\mathfrak{X}))}$$

$$\textsc{null}\ \frac{}{\Gamma \vdash_{\mathfrak{A}} \texttt{null} : \texttt{NullType}_{\emptyset} \,\&\, \alpha(\{\varepsilon\}, \emptyset)}$$

$$\textsc{new}\ \frac{r = \psi(i)}{\Gamma \vdash_{\mathfrak{A}} [\texttt{new}\ C]^i : C_r \,\&\, \alpha(\{\varepsilon\}, \emptyset)}$$

$$\textsc{cast}\ \frac{\Gamma \vdash_{\mathfrak{A}} e : C_R \,\&\, (\mathcal{U}, \mathcal{V}(\mathfrak{X})) \qquad C \preceq D \text{ or } D \preceq C}{\Gamma \vdash_{\mathfrak{A}} (D)\, e : D_R \,\&\, (\mathcal{U}, \mathcal{V}(\mathfrak{X}))}$$

$$\textsc{call}\ \frac{(\tau, \mathcal{U}) = \bigsqcup\{(\tau_r, \mathcal{U}_r) \mid (\tau_r, \mathcal{U}_r, _) = M_{\mathfrak{A}}(m, C_r, \bar{\sigma}) \mid r \in R\}}{\Gamma,\, x : C_R,\, \bar{y} : \bar{\sigma} \vdash_{\mathfrak{A}} x.m(\bar{y}) : \tau \,\&\, (\mathcal{U}, \bigsqcup_{r\in R} X_{m,C_r,\bar{\sigma}})}$$

$$\textsc{get}\ \frac{\tau = \bigsqcup\{F(f, C_r) \mid r \in R\}}{\Gamma,\, x : C_R \vdash_{\mathfrak{A}} x.f : \tau \,\&\, \alpha(\{\varepsilon\}, \emptyset)}$$

$$\textsc{set}\ \frac{\tau \leq: F(f, C_r) \text{ for all } r \in R}{\Gamma, x : C_R, y : \tau \vdash_{\mathfrak{A}} x.f := y : \tau \,\&\, \alpha(\{\varepsilon\}, \emptyset)}$$